

© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2022

V. Sarcar, *Java Design Patterns*

https://doi.org/10.1007/978-1-4842-7971-7_9

9. Decorator Pattern

Vaskaran Sarcar¹

(1) Garia, Kolkata, India

This chapter covers the Decorator pattern, which is also known as the Wrapper pattern.

GoF Definition

It attaches additional responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing for extending functionality.

Concept

An alternative to inheritance is composition. It is quite common in programming and often gives you a better payoff. This chapter shows a useful case study on this topic using some wrappers.

So, what is a wrapper(or a decorator)? **A wrapper is like a topping that surrounds an object. The enclosing object is often called a decorator, which conforms to the interface of the component that it decorates. In programming, you often use a wrapper to add functionalities dynamically.** This is a powerful technique because you can add or discard a wrapper as per your need, and it does not hamper the functionalities of the original object.

Say you need to work on a piece of code and add new features. Someone coded it before and you cannot change the existing code. This scenario is common in software industries when you need to enhance a feature to attract new customers, but you cannot alter the existing workflow of the software to support the existing customers. You understand that in this case, since you were not a part of the team that wrote the first version of the software, you do not have exclusive control from the beginning. Wrappers are useful in similar situations. As said before, in this case, you can add new functionality on top of the existing functionality to support new customers. In fact, by using different types of wrappers you can support different types of customers.

This pattern is most effective when you add decorators dynamically. It's interesting to note since decorators are often added dynamically, it is perfectly fine if you do not want them in a later phase of development, because the original object can still work for you. The upcoming example will make the concept clearer to you.

Real-Life Example

Consider a group of people who each want to build a home. Each person has a different budget and mindset. So, this group visits a home builder to get cost estimates. For simplicity, let's assume that they have the following options:

- They can either make a basic home with standard facilities or they can make an advanced home with additional facilities. For illustration purposes, let's refer to these homes as `BasicHome` and `AdvancedHome`, respectively.
- The builder gives them options of a playground, a swimming pool, or both. Let's call them luxuries. Each luxury means additional cost to the buyer.

Based on their budget, a customer can opt for various options and thus the final price will vary. Most importantly, a customer who opts for a `BasicHome` today can upgrade their home tomorrow by adding a playground or a swimming pool (or both). Each luxury (playground or swimming pool) acts as a decorator in this example.

Computer World Example

The I/O Streams implementations in the .NET Framework, .NET Core, and Java use the concept of the Decorator pattern.

- For example, notice the use of the Decorator pattern in the I/O Streams implementations in both .NET Framework and Java. For example, the `java.io.BufferedOutputStream` class can decorate any `java.io.OutputStream` object.
- Similarly, in the C# programming language, notice the use of the `BufferedStream` class, which inherits from the `Stream` class. If you look closely, you'll notice the presence of two overloaded constructors in this class; each takes a `Stream` (parent class) as a parameter. (This construct is similar to the upcoming demonstration.)

In general, when you see this kind of construct, there is a possibility that you are seeing a Decorator pattern.

Implementation

Let's continue the discussion of the real-life example. Your job is to write a program that mimics a similar case. Before you see the program, let's analyze the different approaches.

Using Subclassing

If you want to provide a solution using inheritance, first try to understand the problems that are associated with it. Assume that you start with the following structure:

```
class Home {  
    // Some code  
}  
class PlayGround extends Home {  
    // Some code  
}  
class SwimmingPool extends PlayGround {  
    // Some code
```

```
}
```

This is not a recommended approach, because to get a swimming pool, you first get a playground, which a customer may not want. Due to similar logic, the following structure is not a good choice either:

```
class Home {  
    // Some code  
}  
class SwimmingPool extends Home {  
    // Some code  
}  
class PlayGround extends SwimmingPool {  
    // Some code  
}
```

This time, to get a playground you first get a swimming pool that a customer may not want. **So, implementing a multilevel inheritance, in this case, is not a good idea!**

Now let's assume that you start with a hierarchical inheritance where `SwimmingPool` and `PlayGround` both inherit from the `Home` class, as shown in Figure [9-1](#).

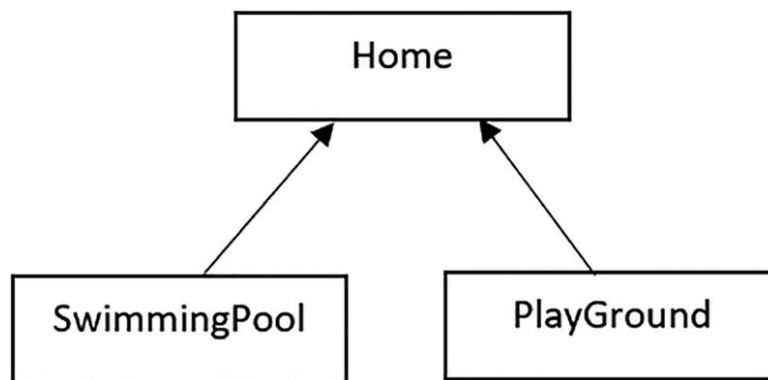


Figure 9-1 A hierarchical inheritance

Now you need a home with a swimming pool and a playground, so you end up with the design shown in Figure [9-2](#).

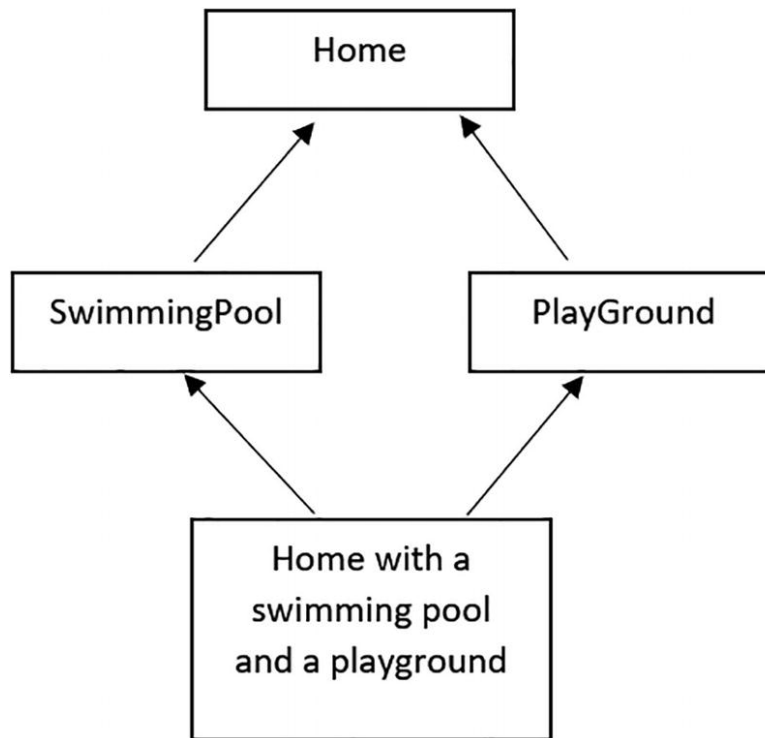


Figure 9-2 A class needs to inherit from multiple base classes. It causes the diamond problem in Java

But you know that **you cannot have multiple base classes in Java**. So, any construct like the following will raise the following compilation error:

```
class Home extends SwimmingPool, PlayGround // Error
{
}

```

Now you understand that using simple subclassing, in this case, is not a good idea. What are the alternatives? Let's continue investigating.

You may proceed with an interface for the luxuries. For example, you can opt for the following interface:

```
interface Luxury {
    void addPlayGround();
    void addSwimmingPool();
}

```

Now you can have a class that can implement this interface. For example, here is a Home class that extends the BasicHome class and implements the Luxury interface:

```
class Home extends BasicHome implements Luxury {
    public void addPlayGround(){
        // Some code
    }
    public void addSwimmingPool(){
        // Some code
    }
}

```

But again, a customer may opt for a home with one of these luxuries, but not both. In that case, if a method is not needed, you write

```
throw new NotImplementedException();

```

The problem associated with this is discussed in the context of the Liskov Substitution Principle (LSP) in Chapter 1. To avoid this, you may follow the **Interface Segregation Principle (ISP)** and segregate the `Luxury` interface. Yes, this time it will work! But in this chapter, let's search for an alternative approach.

Using Object Composition

Let's see how a wrapper can help. **Using a wrapper, you surround an object with another object.** It forwards the requests to the original component and can perform additional operations before or after those requests. **You can add an unlimited number of responsibilities with this concept.** The following figures will help you understand this.

Figure 9-3 shows a home (basic or advanced) surrounded by a playground.

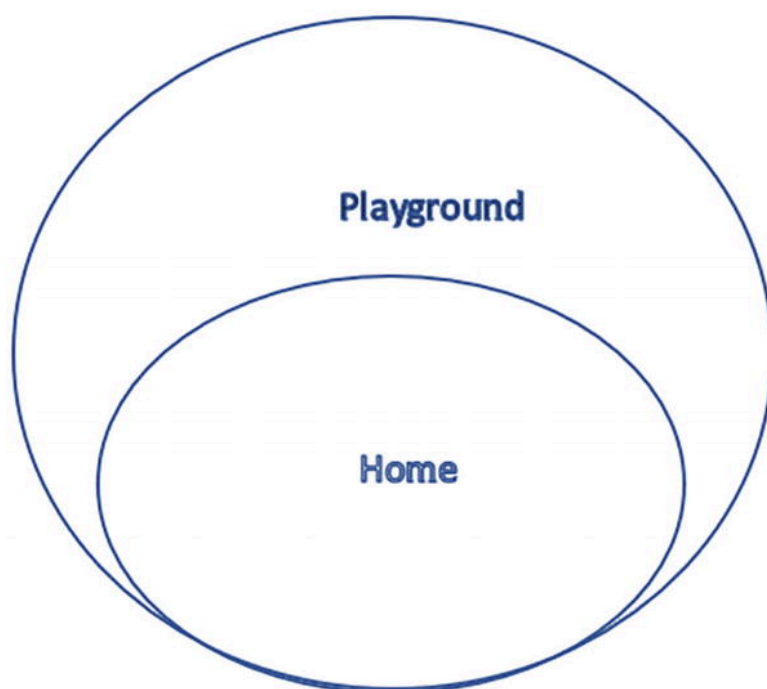


Figure 9-3 The home is surrounded by a playground

Figure 9-4 shows a home surrounded by a swimming pool.

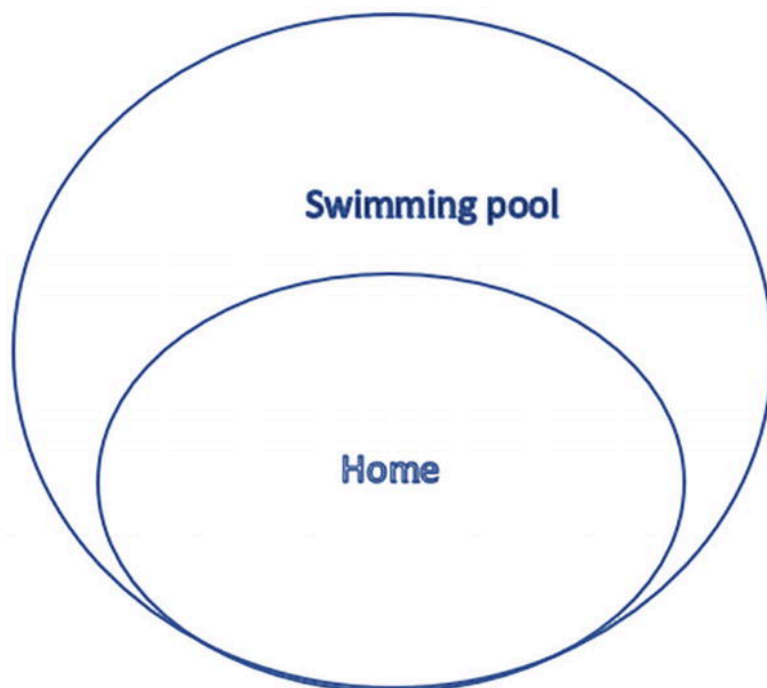


Figure 9-4 The home is surrounded by a swimming pool

Figure [9-5](#) shows the home surrounded by a playground and swimming pool. Here you first surround the home with a playground and then you surround both with a swimming pool.

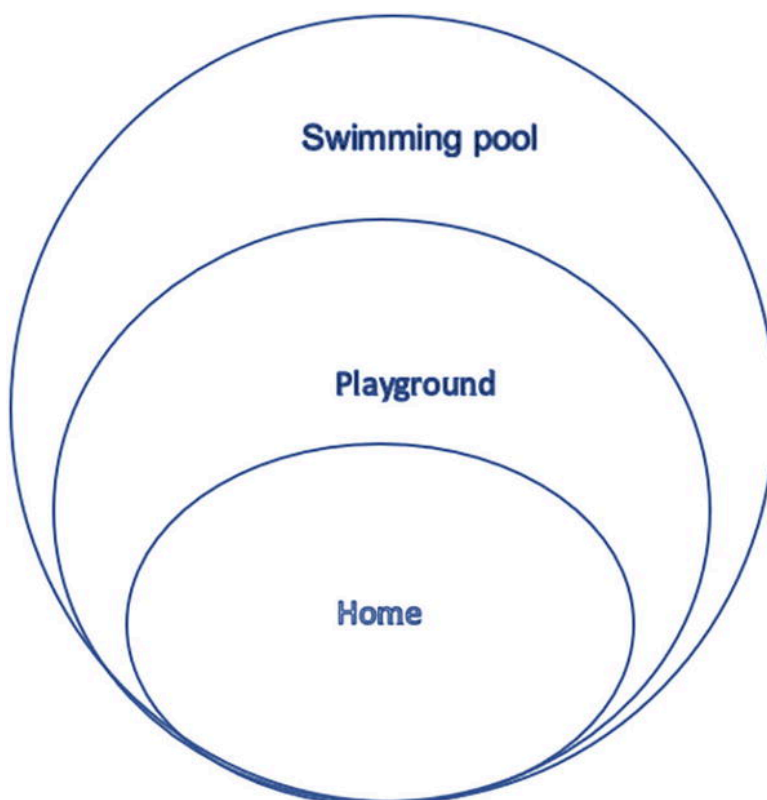


Figure 9-5 The home is surrounded by a playground and a swimming pool

Figure [9-6](#) shows the home surrounded by a swimming pool and a playground again. But this time you change the order; you first surround the home with a swimming pool, and then you surround both with a playground.

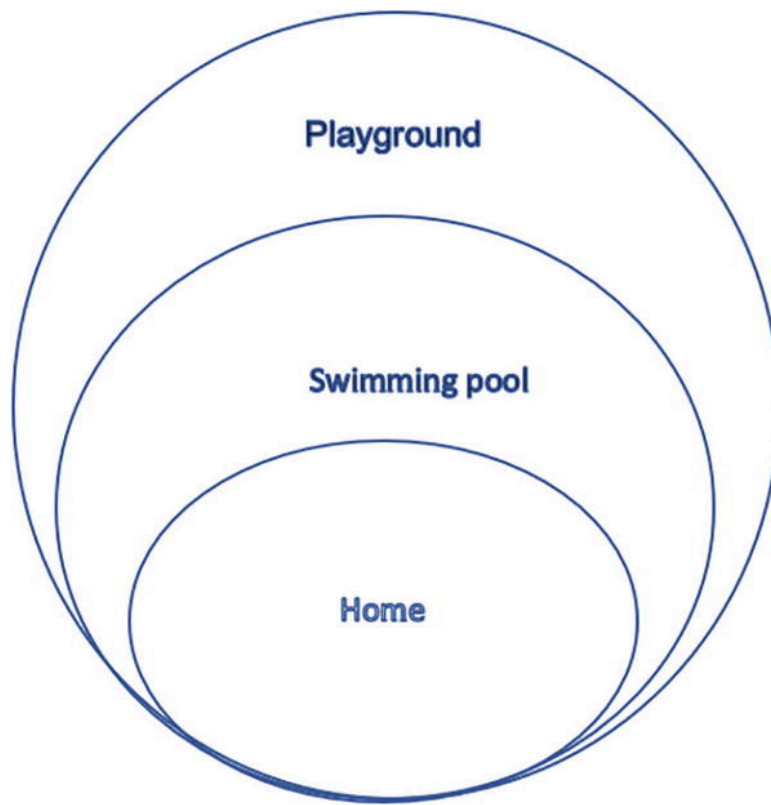


Figure 9-6 The home is surrounded by a swimming pool. Later you surround both with a playground

Following the same technique, you can add two more playgrounds or swimming pools.

Let's implement this concept following the requirements you have. In the upcoming demonstration, six players are involved: `Home`, `BasicHome`, `AdvancedHome`, `Luxury`, `PlayGround`, and `SwimmingPool`. The first three form one inheritance hierarchy and the last three are created to add new functionalities.

The `Home` class is defined as follows:

```
abstract class Home {
    public double basePrice;
    public double additionalCost;
    public Home() {
        // Minimum home price is $100000
        this.basePrice = 100000.0;
        this.additionalCost = 0.0;
    }
    public abstract double getPrice();
}
```

Here are some important points:

- You can see that a concrete implementor of `Home` must implement the `getPrice()` methods. In this example, `BasicHome` and `AdvancedHome` inherit from `Home`. So, they override the `getPrice()` method as per their needs.
- Assume that the base price of a home is \$100,000. Using the `additionalPrice` attribute, set an extra price for the advanced home. Currently, for a basic home, this cost is \$0 and for an advanced home, this cost is \$25,000.

- Assume that once the home is built, there is no need for an immediate modification. One can add the luxuries later.
- Once a home is built, you can opt for a playground or a swimming pool for an existing home, or you may want both. So, the `PlayGround` and `SwimmingPool` classes appear in this example. In the client class, there are eight different scenarios to demonstrate the power of this design pattern. There you see a client can opt for different homes with different facilities in different orders.
- Although it was not strictly needed, to share the common code, both the `Playground` class and `SwimmingPool` class inherit from the abstract class `Luxury`, which has the following structure:

```
abstract class Luxury extends Home {
    protected Home home;
    public double luxuryCost;
    public Luxury(Home home) {
        this.home = home;
    }
    @Override
    public double getPrice() {
        return home.getPrice();
    }
}
```

- Like the `additionalPrice` attribute, you can set a luxury cost using the `luxuryCost` attribute. You use it inside the constructors.
- Notice that `Luxury` holds a reference of `Home`. So, the concrete decorators (`PlayGround` or `SwimmingPool` in this example) can decorate an instance of `Home`.
- Now let's look into the structure of a concrete decorator, say `PlayGround`, which is as follows:

```
class PlayGround extends Luxury {
    public PlayGround(Home home) {
        super(home);
        this.luxuryCost = 20000;
        System.out.println(" For a playground, you pay an
                           extra $" + this.luxuryCost);
    }
    @Override
    public double getPrice() {
        return home.getPrice() + luxuryCost;
    }
}
```

- You can see that when you get this facility, you pay an additional \$20,000. You initialize this value inside the constructor.
- Most importantly, notice that how the final price is calculated. It calls a `Home` instance price and then adds the luxury cost. The original `Home` hierarchy is not touched.
- The `SwimmingPool` class works similarly, but you have to pay more for this. (Yes, I assume that a swimming pool costs more than a playground in this case).

Class Diagram

Figure 9-7 shows the most important parts of the class diagram.

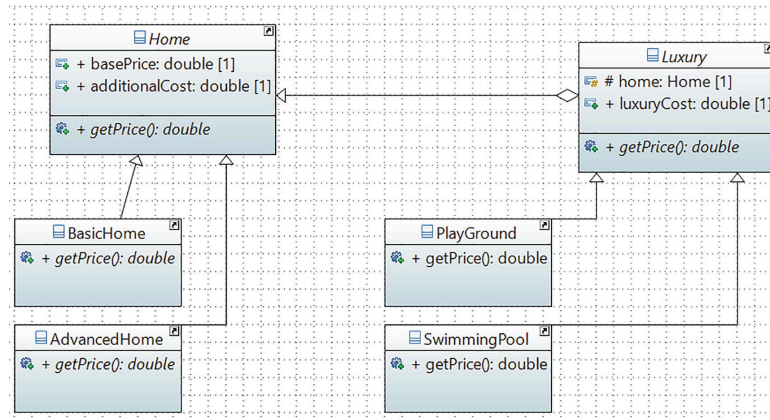


Figure 9-7 Class diagram (without the client class)

Package Explorer View

Figure 9-8 shows the high-level structure of the program.

decorator

- ▼  AdvancedHome.java
 - ▼  AdvancedHome
 -  AdvancedHome()
 -  getPrice() : double
- ▼  BasicHome.java
 - ▼  BasicHome
 -  BasicHome()
 -  getPrice() : double
- >  Client.java
- ▼  Home.java
 - ▼  Home
 - additionalCost
 - basePrice
 -  Home()
 -  getPrice() : double
- ▼  Luxury.java
 - ▼  Luxury
 - ◊ home
 - luxuryCost
 -  Luxury(Home)
 -  getPrice() : double
- ▼  PlayGround.java
 - ▼  PlayGround
 -  PlayGround(Home)
 -  getPrice() : double

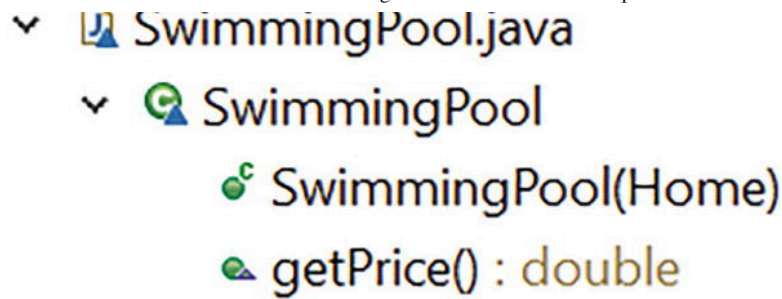


Figure 9-8 Package Explorer view

Demonstration

Here is the complete demonstration for you. In the client code, you see many different scenarios to show the effectiveness of this application. All parts of the program can be seen inside the package `jdp3e.decorator`.

```
// Home.java
abstract class Home {
    public double basePrice;
    public double additionalCost;
    public Home() {
        // Minimum home price is $100000
        this.basePrice = 100000.0;
        this.additionalCost = 0.0;
    }
    public abstract double getPrice();
}

// BasicHome.java
class BasicHome extends Home {
    public BasicHome() {
        // No additional cost for a basic home.
        System.out.println(" The basic home with some
            standard facilities are ready.");
        System.out.println(" You need to pay $" +
            this.getPrice() + " for this.");
    }
    @Override
    public double getPrice() {
        // There is no additional cost for a basic home
        // So, returning the basic price is sufficient.
        return basePrice;
    }
}

// AdvancedHome.java
class AdvancedHome extends Home {

    public AdvancedHome() {
        additionalCost = 25000;
        System.out.println(" It becomes an advanced home
            with more facilities.");
        System.out.println(" You need to pay $" +
            this.getPrice() + " for this.");
    }
    @Override
    public double getPrice() {
        return basePrice + additionalCost;
    }
}

// Luxury.java
```

```

abstract class Luxury extends Home {
    protected Home home;
    public double luxuryCost;
    public Luxury(Home home) {
        this.home = home;
    }
    @Override
    public double getPrice() {
        return home.getPrice();
    }
}

// Playground.java
class Playground extends Luxury {
    public Playground(Home home) {
        super(home);
        this.luxuryCost = 20000;
        System.out.println(" For a playground, you pay an
                               extra $" + this.luxuryCost);
    }
    @Override
    public double getPrice() {

        return home.getPrice() + luxuryCost;
    }
}

// SwimmingPool.java
class SwimmingPool extends Luxury {
    public SwimmingPool(Home home) {
        super(home);
        this.luxuryCost = 55000;
        System.out.println(" For a swimming pool, you pay
                               an extra $" + this.luxuryCost);
    }
    @Override
    public double getPrice() {
        return home.getPrice() + luxuryCost;
    }
}

// Client.java
class Client {
    public static void main(String[] args) {
        System.out.println("***Using wrappers in different scenarios.***\n");
        System.out.println("Scenario-1: Making a basic home
                               with standard facilities.");
        Home home = new BasicHome();
        System.out.println("Total cost: $" +
                               home.getPrice());
        System.out.println("\nScenario-2: Making a basic
                               home. Then adding a playground.");
        home = new BasicHome();
        home = new Playground(home);
        System.out.println("Total cost: $" +
                               home.getPrice());
        System.out.println("\nScenario-3: Making a basic
                               home. Then adding two
                               playgrounds one-by-one.");
        home = new BasicHome();
        home = new Playground(home);
    }
}

```

```

        home = new PlayGround(home);
        System.out.println("Total cost: $" +
                            home.getPrice());
        System.out.println("\nScenario-4: Making a basic
                            home. Then adding one additional
                            playground and swimming pool.");

        home = new BasicHome();
        home = new PlayGround(home);
        home = new SwimmingPool(home);
        System.out.println("Total cost: $" +
                            home.getPrice());
        System.out.println("\nScenario-5: Adding a swimming
                            pool and then a playground
                            to a basic home.");

        home = new BasicHome();
        home = new SwimmingPool(home);
        home = new PlayGround(home);
        System.out.println("Total cost: $" +
                            home.getPrice());
        System.out.println("\nScenario-6: Making an
                            advanced home now.");

        home = new AdvancedHome();
        System.out.println("Total cost: $" +
                            home.getPrice());
        System.out.println("\nScenario-7: Making an
                            advanced home. Then adding one
                            additional playground to it.");

        home = new AdvancedHome();
        home = new PlayGround(home);
        System.out.println("Total cost: $" +
                            home.getPrice());
        System.out.println("\nScenario-8: Making an
                            advanced home. Then adding one

                            additional playground and one swimming
                            pool to it.");

        home = new AdvancedHome();
        home = new PlayGround(home);
        home = new SwimmingPool(home);
        System.out.println("Total cost: $" +
                            home.getPrice());
    }
}

```

Output

```

***Using wrappers in different scenarios

. ***
Scenario-1: Making a basic home with standard facilities.
The basic home with some standard facilities are ready.
You need to pay $100000.0 for this.
Total cost: $100000.0
Scenario-2: Making a basic home. Then adding a playground.
The basic home with some standard facilities are ready.
You need to pay $100000.0 for this.
For a playground, you pay an extra $20000.0

```

Total cost: \$120000.0

Scenario-3: Making a basic home. Then adding two playgrounds one by one.

The basic home with some standard facilities are ready.

You need to pay \$100000.0 for this.

For a playground, you pay an extra \$20000.0

For a playground, you pay an extra \$20000.0

Total cost: \$140000.0

Scenario-4: Making a basic home. Then adding one additional playground and swimming po

The basic home with some standard facilities are ready.

You need to pay \$100000.0 for this.

For a playground, you pay an extra \$20000.0

For a swimming pool, you pay an extra \$55000.0

Total cost: \$175000.0

Scenario-5: Adding a swimming pool and then a playground to basic home.

The basic home with some standard facilities are ready.

You need to pay \$100000.0 for this.

For a swimming pool, you pay an extra \$55000.0

For a playground, you pay an extra \$20000.0

Total cost: \$175000.0

Scenario-6: Making an advanced home now.

It becomes an advanced home with more facilities.

You need to pay \$125000.0 for this.

Total cost: \$125000.0

Scenario-7: Making an advanced home. Then adding one additional playground to it.

It becomes an advanced home with more facilities.

You need to pay \$125000.0 for this.

For a playground, you pay an extra \$20000.0

Total cost: \$145000.0

Scenario-8: Making an advanced home. Then adding one additional playground and one sw

It becomes an advanced home with more facilities.

You need to pay \$125000.0 for this.

For a playground, you pay an extra \$20000.0

For a swimming pool, you pay an extra \$55000.0

Total cost: \$200000.0

Q&A Session

9.1 You say that composition promotes dynamic behavior in this design. You also say that it is better than inheritance in this example.

Can you throw some more light on this discussion?

When a derived class inherits from a base class, it actually inherits the behavior of the base class at that time only. Although different subclasses can extend the parent class in different ways, this type of binding is known at compile time. So, this method is static in nature. But by using the concept of composition, as in the previous example, you implement dynamic behaviors.

Notice carefully: a decorator object is designed in such a way that it has the same interface as the underlying object. It allows a client object to interact with the decorated object in the same way as it would with the underlying object. This is the reason: a decorator object contains a reference to the actual object. When a client sends a request, the decorated objects forward this request to the underlying object. But it can add functionality before or after forwarding the request to the actual object. This ensures

that you add behavior at runtime without modifying the original object structure.

When you design a parent class, you may not have enough visibility about what kind of additional responsibilities your clients may want in some later phase. Since the constraint is that you cannot modify the existing code, in this case, object composition not only outclasses inheritance but also ensures that you are not introducing bugs in the old architecture.

Lastly, in this context, you must try to remember one key design principle that says *classes should be open for extension but closed for modification*.

9.2 What are the key advantages of using a decorator?

Here are some key advantages:

- The existing structure is untouched, so you do not introduce bugs in that part of the code.
- New functionalities can be added to an existing object easily.
- You can not only add a behavior to an interface, but you can alter the behavior too.
- You do not need to predict/implement all the supported functionalities at once (for example, in the initial design phase). You can develop incrementally. For example, you can add decorator objects one by one to support your needs. If you make a complex class first and then want to extend the functionalities, that will be a tedious process.

9.3 How is the overall design pattern different from inheritance?

You can add or alter or remove responsibilities by simply attaching or detaching decorators. But with simple inheritance techniques, you need to create new classes for new responsibilities. So, it is possible that you will end up with a complex system. In this context, read the discussion of the subtopic “[Using Subclassing](#)” under “Implementation” in this chapter. You can also revisit Q&A 9.1 in this context.

9.4 I notice that you have created the classes with a single responsibility. Is this beneficial?

Yes. It makes the Decorator pattern more efficient because you can simply add or remove a responsibility dynamically one at a time.

9.5 What are the disadvantages associated with this pattern?

I believe that if you are careful, there are no significant disadvantages. But if you create too many decorators in the system, it can be hard to maintain and debug. So, in that case, they create unnecessary complexities.

9.6 In the example, the `Luxury` class is abstract, but there is no abstract method in it. How is this possible?

In Java, you can have an abstract class without any abstract method in it, but the reverse is not true. So, if a class contains at least one abstract method, it means that the class itself is incomplete and you are forced to mark it with `abstract` keyword.

Let's revisit the `Luxury` class and notice the bold line:

```
abstract class Luxury extends Home {
    protected Home home;
    public double luxuryCost;
    public Luxury(Home home) {
        this.home = home;
    }
    @Override
    public double getPrice() {
        return home.getPrice();
    }
}
```

In the bold line of code, you delegate a task. In addition, you want to use and instantiate the concrete decorators only. So, in this example, you cannot simply instantiate the `Luxury` instance because it is marked with the `abstract` keyword.

9.7 Is it mandatory to use decorators for dynamic binding only?

No. You can use the concept for both static and dynamic binding. But dynamic binding is its strength, so I concentrated on that here. The GoF definition also focuses on dynamic binding only.

9.8 In simple words, you are using decorators to wrap the core architecture. Is this understanding correct?

Yes. These are wrapper codes to extend the core functionalities of the application. But the original architecture is untouched when you use them.

9.9 Can you throw some light on how `BufferedOutputStream` plays the role of a decorator?

Consider the following code segment, which we often see:

```
import java.io.BufferedOutputStream;
import java.io.FileNotFoundException;
import java.io.FileOutputStream;
import java.io.OutputStream;
class JDKExample {
    public static void main(String[] args) throws
        FileNotFoundException {
        OutputStream stream;
        stream = new FileOutputStream("somefile.txt");
        stream =new BufferedOutputStream(stream);
        // Some other code
    }
}
```

Now investigate further. You see the following:

```
public class BufferedOutputStream extends FilterOutputStream {
    public BufferedOutputStream(OutputStream out) {
        this(out, 8192);
    }
    // Remaining code skipped
}
```



```
}

```

It means the `BufferedOutputStream` inherits from `FilterOutputStream`.
Now look into this class:

```
public class FilterOutputStream extends OutputStream {
    /**
     * The underlying output stream to be filtered.
     */
    protected OutputStream out;
    public FilterOutputStream(OutputStream out) {
        this.out = out;
    }
    // remaining code skipped
}
```

You also see that the `FileOutputStream` class extends the `OutputStream` class. Here is a sample code segment:

```
public class FileOutputStream extends OutputStream {
    // Remaining code skipped
}
```

So, the overall structure can be simplified with the diagram in Figure 9-9.

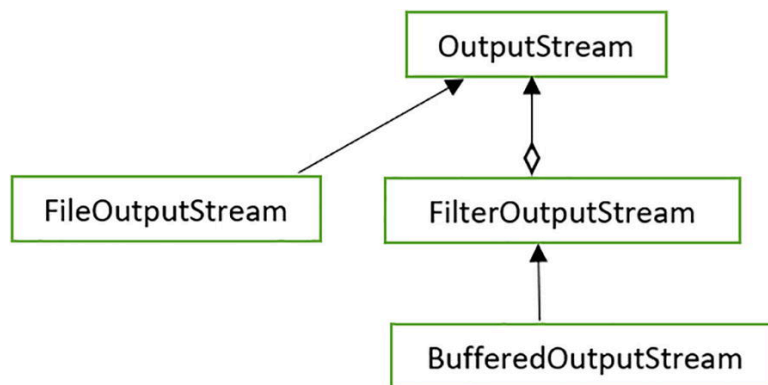


Figure 9-9 A hierarchical inheritance

Now you understand the following:

- The `Luxury` class plays the role of `FilterOutputStream` (an abstract decorator).
- The `PlayGround` and `SwimmingPool` classes play the role of `BufferedOutputStream` (a concrete decorator).
- The `Home` class plays the role of the `OutputStream` (the abstract class).
- `BasicHome` and `AdvancedHome` play the role of `FileOutputStream` (the concrete class that inherits from the abstract class).

[Previous chapter](#)

[Next chapter](#)

[< 8. Proxy Pattern](#)

[10. Adapter Pattern >](#)